

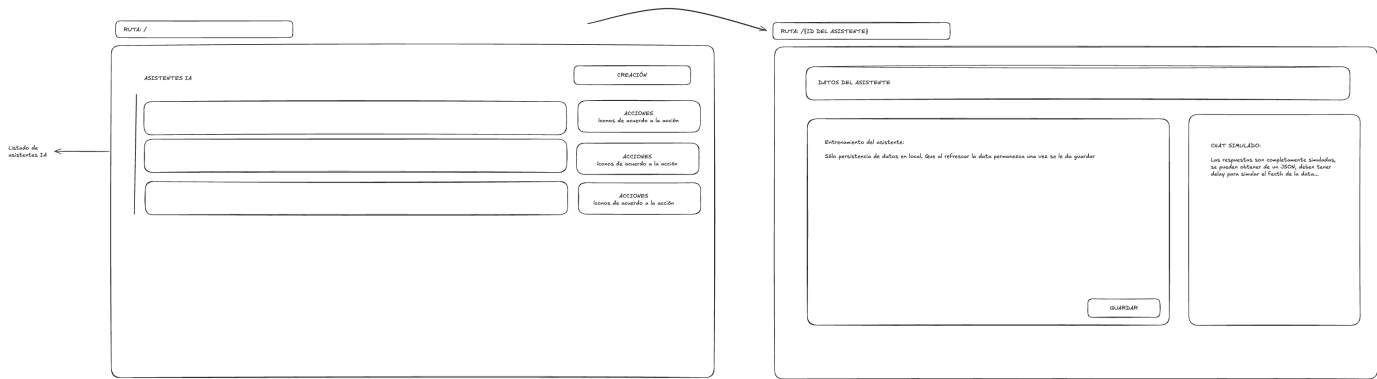
Módulo de Gestión de Asistentes IA

Contexto

Funnelhot está desarrollando un sistema de asistentes IA para automatizar interacciones con leads. Tu tarea es crear el módulo de gestión de estos asistentes.

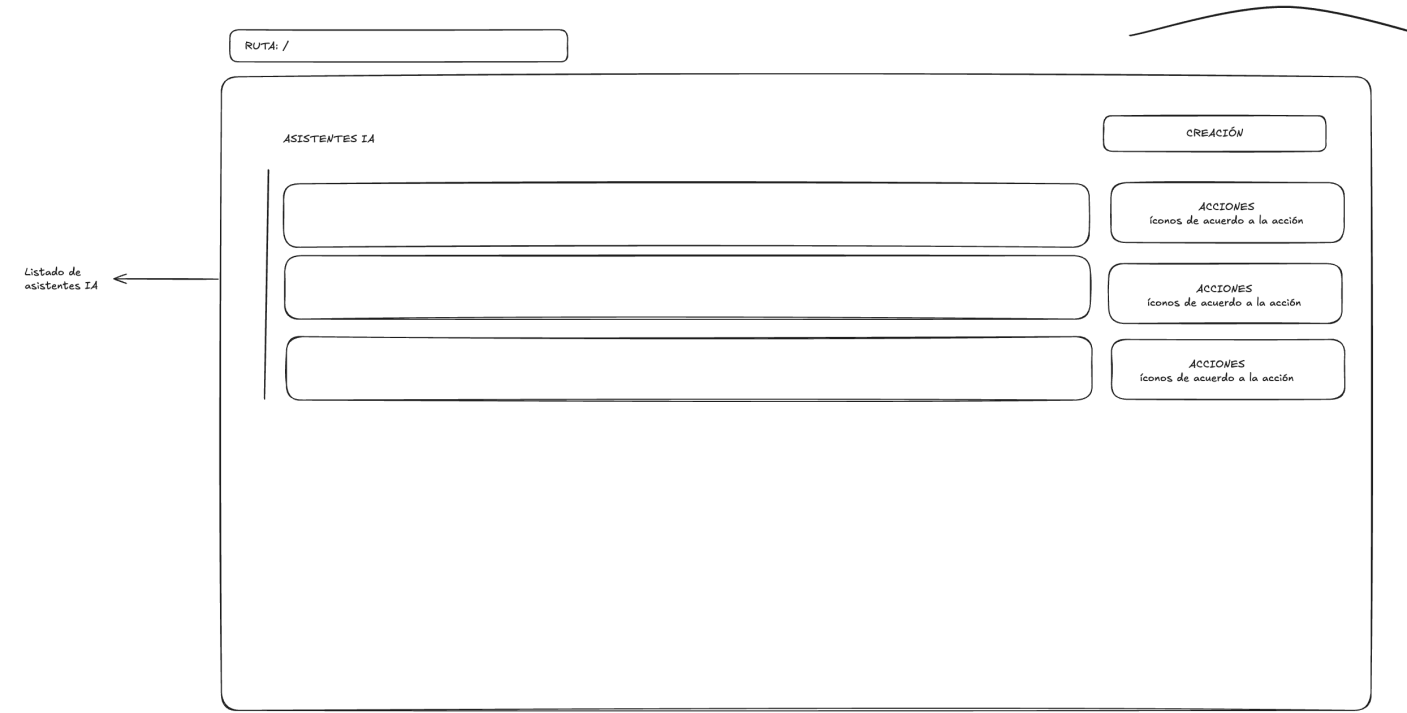
Objetivo

Desarrollar una aplicación web **responsive** que permita crear, listar, editar, eliminar y entrenar asistentes de IA, con persistencia local de datos.



Requisitos Funcionales

1. Página Principal (Listado de Asistentes)



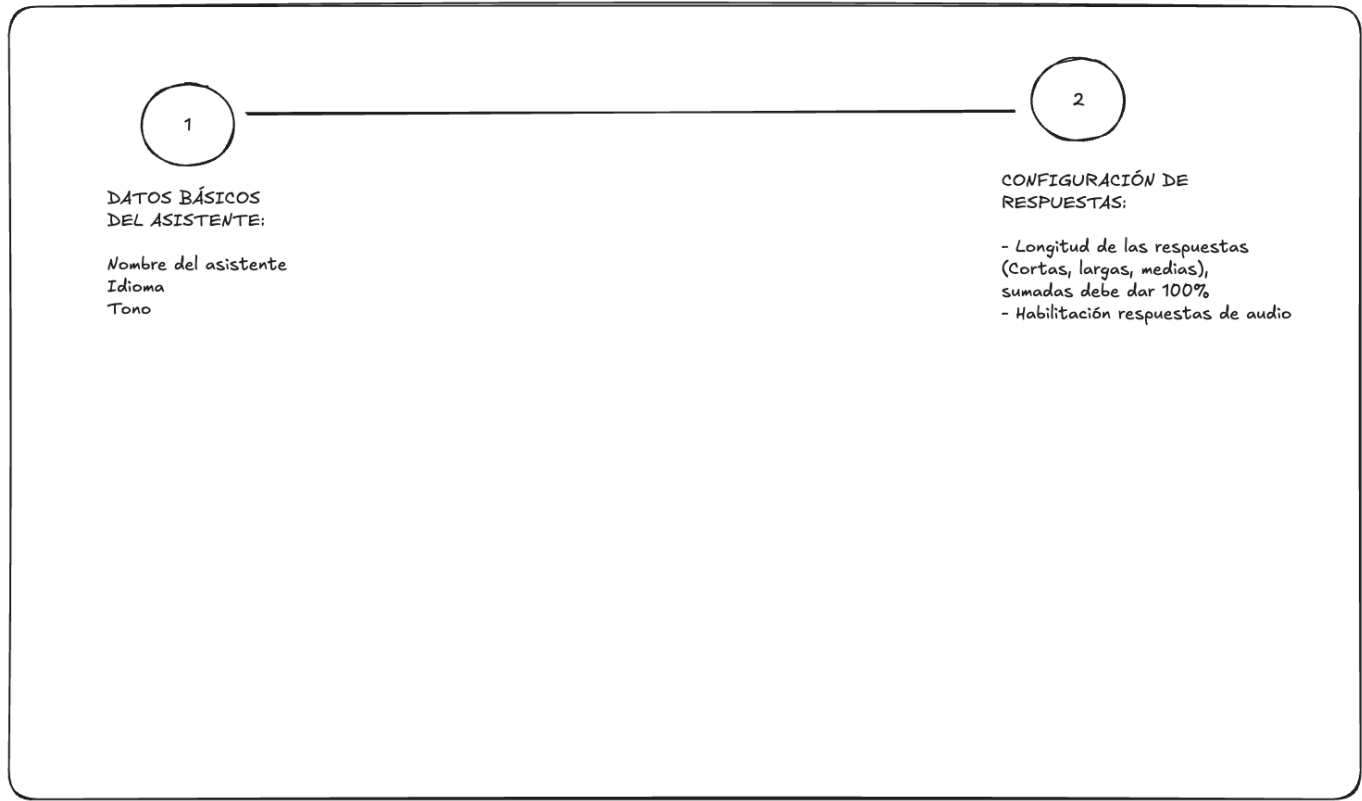
Ruta: /

- Mostrar listado de asistentes existentes en formato de tarjetas
- Cada tarjeta debe mostrar:
 - Nombre del asistente
 - Idioma
 - Tono/personalidad
 - Menú de acciones (Editar, Eliminar, Entrenar)
- Botón "Crear Asistente" que abra el modal de creación

- Si no hay asistentes, mostrar un estado vacío apropiado

2. Modal de Creación/Edición

MODAL DE CREACIÓN/EDICIÓN: Se abre al dar clic en los íconos correspondientes. El modal es de dos pasos al dar clic en siguiente se habilita el paso siguiente. Debe incluir validaciones.



Comportamiento: Modal de 2 pasos

Paso 1: Datos Básicos

- Nombre del asistente (requerido, mínimo 3 caracteres)
- Idioma (requerido, select con opciones: Español, Inglés, Portugués)
- Tono (requerido, select con opciones: Formal, Casual, Profesional, Amigable)
- Botón "Siguiente" que valide y habilite el paso 2

Paso 2: Configuración de Respuestas

- Longitud de respuestas (requerido, respuestas: Cortas, Medias, Largas, la persona puede indicar el porcentaje de cada una de esas respuestas. La suma de short + medium + long debe ser **100**). Ejemplo: Si short=30, medium=50, long=20, significa que el 30% de las respuestas serán cortas, 50% medianas, 20% largas.
- Habilitar respuestas de audio (checkbox, opcional)
- Indicador visual del paso actual (1 o 2)
- Botones "Atrás" y "Guardar"

Validaciones:

- Todos los campos marcados como requeridos deben validarse
- Mostrar mensajes de error claros (idealmente con un alert)
- La suma de longitudes de las respuestas debe ser 100%
- No permitir avanzar al paso 2 sin completar el paso 1

3. Página de Entrenamiento

RUTA: /{ID DEL ASISTENTE}

DATOS DEL ASISTENTE

Entrenamiento del asistente:

Sólo persistencia de datos en local. Que al refrescar la data permanezca una vez se le da guardar

GUARDAR

CHAT SIMULADO:

Las respuestas son completamente simuladas, se pueden obtener de un JSON, deben tener delay para simular el fetch de la data...

Ruta: /{id} (donde id es el identificador del asistente)

- Mostrar información del asistente en la parte superior
- Sección de "Entrenamiento" con:
 - Área de texto para ingresar prompts/instrucciones
 - Botón "Guardar" que simule guardar el entrenamiento
 - Mensaje de éxito al guardar
 - Los datos del entrenamiento persisten en localStorage
- Sección de "Chat Simulado" con:
 - Interfaz de chat simple (mensajes del usuario y del asistente)
 - Input para enviar mensajes
 - Botón para reiniciar la conversación
 - Las respuestas del asistente deben ser simuladas con un delay de 1-2 segundos
 - Obtener respuestas de un JSON local o array predefinido

4. Funcionalidad de Eliminación

- Mostrar confirmación antes de eliminar
- Mensaje de éxito tras eliminar
- Actualizar la lista inmediatamente

Requisitos Técnicos

Obligatorios

- **Framework:** Next.js con App Router
- **Lenguaje:** TypeScript
- **Persistencia:** LocalStorage
- **Diseño:** Responsive (mobile y desktop)
- **Manejo de estados:**
 - Loading states durante operaciones
 - Estados de error apropiados
 - Validaciones en tiempo real
- **Código:**
 - Componentes reutilizables

- Estructura de carpetas clara y escalable
- Nombres descriptivos para variables y funciones
- Comentarios donde sea necesario

Diseño

Se adjuntan wireframes de referencia para:

1. Página principal con listado
2. Modal de creación/edición (2 pasos)
3. Página de entrenamiento

Diseño

Se adjuntan wireframes de referencia para:

1. Página principal con listado
2. Modal de creación/edición (2 pasos)
3. Página de entrenamiento

Nota sobre el diseño

Lo que esperamos:

Los wireframes son **guías estructurales**, no diseños pixel-perfect. Se espera que el desarrollador frontend tome decisiones que resulten en una interfaz profesional y funcional:

- **Espaciados y márgenes:** Que la interfaz respire y sea cómoda de usar
- **Paleta de colores:** Elige colores que funcionen bien juntos y sean accesibles
- **Tipografía:** Jerarquía clara, tamaños legibles, pesos apropiados
- **Iconografía:** Usa iconos donde mejoren la comprensión (puedes usar librerías gratuitas como React Icons, Heroicons, Lucide, etc.)
- **Estados interactivos:** Hover, focus, active, disabled - la interfaz debe sentirse responsiva
- **Feedback visual:** Animaciones sutiles, transiciones suaves

Lo que valoramos:

- Interfaz limpia y moderna
- Experiencia de usuario intuitiva
- Consistencia visual en toda la aplicación
- Que se note que pensaste en los detalles

Lo que NO buscamos:

- Diseño sobrecargado o con demasiados efectos
- Paletas de colores difíciles de leer o poco profesionales

Muéstranos que sabes tomar un requerimiento básico y convertirlo en algo que se vea bien y funcione bien, tal como harías en el día a día del trabajo.

Datos de Ejemplo

Para facilitar las pruebas, si lo deseas, puedes inicializar la aplicación con estos datos:

```
[
  {
    "id": "1",
    "name": "Asistente de Ventas",
    "language": "Español",
    "tone": "Profesional",
    "responseLength": {
      "short": 30,
      "medium": 50,
      "long": 20
    },
    "audioEnabled": true,
    "rules": "Eres un asistente especializado en ventas. Siempre sé cordial y enfócate en identificar necesidades del cliente antes de ofrecer productos."
  },
  {
    "id": "2",
```

```
    "name": "Soporte Técnico",
    "language": "Inglés",
    "tone": "Amigable",
    "responseLength": {
      "short": 20,
      "medium": 30,
      "long": 50
    },
    "audioEnabled": false,
    "rules": "Ayudas a resolver problemas técnicos de manera clara y paso a paso. Siempre confirma que el usuario haya entendido antes de continuar."
  },
]
```

Respuestas simuladas para el chat (puedes agregar más):

```
[
  "Entendido, ¿en qué más puedo ayudarte?",
  "Esa es una excelente pregunta. Déjame explicarte...",
  "Claro, con gusto te ayudo con eso.",
  "¿Podrías darme más detalles sobre tu consulta?",
  "Perfecto, he registrado esa información."
]
```

Entrega

Qué entregar

1. Repositorio de GitHub (público o privado con acceso)
2. README con:
 - Instrucciones para correr el proyecto
 - Decisiones técnicas que tomaste y por qué
 - Características implementadas
 - Si tuviste que priorizar, qué dejaste fuera y por qué
 - Tiempo aproximado de dedicación

Criterios de Evaluación

1. **Funcionalidad**
 - Todas las funcionalidades requeridas funcionan correctamente
 - Manejo apropiado de errores
 - Persistencia de datos
2. **Código**
 - Estructura y organización
 - Uso correcto de TypeScript
 - Componentes reutilizables
 - Convenciones de nomenclatura
3. **UI/UX**
 - Diseño responsive
 - Experiencia de usuario fluida
 - Estados de carga y error claros
 - Creatividad en el diseño
4. **Extras**
 - README bien documentado
 - Detalles que mejoren la experiencia